SIMATS ENGINEERING

CO2_AT3 Error Analysis Task

COURSE NAME:Software Testing **COURSE CODE: CSA3712**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 20%

Sl. No	Questions	Bloom's Level
1	A tester designs test cases for an online shopping system using equivalence partitioning but incorrectly groups input ranges (e.g., valid and invalid inputs mixed in one class). Additionally, boundary values are not considered. Analyze the given test design and identify all errors in equivalence class partitioning and boundary value analysis. Examine the root causes of these mistakes with proper reasoning. Propose corrected test cases with proper classification and boundary coverage. Justify how your corrections improve defect detection effectiveness.	3
2	A QA engineer performs random testing on a banking application and claims high coverage without using any structured techniques. Critical defects in transaction limits and boundary conditions are missed. Evaluate the testing approach and identify errors in strategy and coverage assumptions. Analyze the root causes behind missing critical defects. Suggest appropriate black box testing techniques to correct the approach. Justify how these techniques ensure better test adequacy and reliability.	3
3	A developer performs white box testing but creates an incomplete control flow graph, missing decision nodes and loops, resulting in poor path coverage. Analyze the errors in the control flow graph and path testing approach. Identify the root causes for inadequate test coverage. Propose corrections by constructing a proper control flow representation and identifying all independent paths. Justify how improved path testing ensures better coverage and defect detection.	5

4	A testing team reports 100% statement coverage for a module but fails to detect logical errors in conditional statements. Evaluate the adequacy of the testing performed and identify gaps in coverage criteria. Analyze why statement coverage alone was insufficient. Recommend improved white box testing strategies such as branch or path coverage. Justify how the revised approach enhances overall test effectiveness and error detection.	5
5	A tester uses Equivalence Class Partitioning for an age-validation field (0-100) but defines only two classes: Negative numbers and Positive numbers. Analyze the errors in this partitioning strategy. Identify why this fails to account for valid vs. invalid age limits and the absence of Boundary Value Analysis. Propose a corrected set of partitions and justify how they ensure more robust input validation.	3
6	A developer claims full Condition Coverage for a complex `if(A && B	
7	A QA team applies Decision Table Testing for a discount system but misses several combinations of "Premium Member" and "Coupon Applied," leading to price calculation errors. Analyze the errors in the construction of the decision table. Identify why the current approach is incomplete. Propose a revised, exhaustive decision table and justify its effectiveness in preventing production defects.	3
8	During unit testing of a looping structure, a tester checks only the first and last iterations (n=1 and n=max). Defects occurring at n=0 and n=max+1 are missed. Evaluate the errors in this Loop Testing strategy. Analyze the root causes behind these omissions. Recommend a corrected approach for testing simple and nested loops, and justify how it ensures path integrity.	5
9	An engineer utilizes Error Guessing as the primary testing technique for a safety-critical braking system, ignoring structured black box methods. Identify the errors in relying solely on ad-hoc techniques for high-risk modules. Analyze the impact on test adequacy. Suggest a combination of BVA and State Transition Testing to fix the strategy, justifying the increased safety assurance.	5

10	A tester designs a Control Flow Graph (CFG) for a module but fails to represent the "else" branches of multiple nested if statements, leading to 100% statement coverage but 0% branch coverage on error paths. Analyze the errors in graph construction. Identify the root causes for the missing nodes. Propose a corrected CFG and independent paths to ensure branch adequacy.	5

ANSWERS

1. Online Shopping System (ECP and BVA Errors)

The test design contains two main errors. First, the equivalence classes are not properly formed, because valid and invalid inputs are mixed in a single class (for example, a quantity class of -5 to 50 that includes both negative and acceptable values). An equivalence class must contain values that the system treats in the same way; when a class mixes valid and invalid inputs, one representative value cannot stand for the whole class, so the result of a test says nothing about the other values. Second, boundary values were not considered at all, so values at the edges of the valid range (such as 0, 1, the maximum order quantity and the maximum plus one), where off-by-one faults such as using > instead of >= are most common, were never exercised.

The root causes are a weak understanding of the requirement (the valid range was not stated clearly before partitioning), partitioning done on the basis of the input type rather than of the system's behaviour, the absence of a review of the classes for overlap and completeness, and the belief that ECP alone is adequate and makes boundary testing unnecessary. In addition, invalid classes were not split by the kind of error (negative, zero, non-numeric, above the maximum), so different failures were hidden inside one class.

The corrected design is as follows. For an order quantity valid from 1 to 10, the classes are: valid (1 to 10), invalid below range (zero or negative), invalid above range (more than 10), and invalid type (decimal, text, empty). The corrected test cases are quantity 5 (valid), -3 (invalid low), 15 (invalid high), "abc" (invalid type) and an empty field. The boundary values are 0, 1, 2, 9, 10 and 11, with expected results of reject, accept, accept, accept, accept and reject respectively. The same method is applied to the payment amount, the coupon code and the pin code fields. These corrections improve defect detection because each class now has a single, well-defined expected behaviour, so one test per class is a reliable representative; and the boundary tests catch the relational-operator and off-by-one defects which random representative values miss, giving broader coverage with a small and efficient test set.

2. Random Testing in a Banking Application

The QA engineer's approach is unsafe because random testing selects inputs without any relation to the specification or to the risk of the system. The assumption that a large number of tests means high coverage is wrong: coverage must be measured against requirements, input partitions and code structure, and random values mostly fall into the commonly used middle of the input domain, repeating the same behaviour many times. The special values that matter in banking, such as the minimum and maximum transfer amount, the daily transaction limit, a balance exactly equal to the withdrawal, and a zero or negative amount, have a very low probability of being selected randomly. Moreover, since no record of the technique or of the expected results was kept, the testing is neither repeatable nor measurable, and the claim of high coverage cannot be verified.

The root causes of the missed critical defects are therefore the lack of a test design technique, no analysis of the requirements to find the limits and rules, no traceability between tests and requirements, no risk-based prioritisation of the financial functions, and overconfidence in the number of tests executed. Defects in the transaction limits and boundary conditions occur at specific points and need deliberate, targeted inputs.

The approach should be corrected by applying structured black-box techniques. Equivalence Class Partitioning divides the amount into valid and invalid classes (for example, a valid amount from 1 to the per-transaction limit, an amount above the limit, zero or negative, and non-numeric). Boundary Value Analysis then tests the edges: 0, 1, limit minus 1, the limit and limit plus 1, and the same for the daily limit. Decision table testing is added for combinations of conditions such as account type, balance and limit, and state transition testing for account states such as active, blocked and frozen. These techniques ensure better adequacy because every requirement, partition and boundary is covered systematically and can be traced and measured, and they ensure reliability because the results are repeatable and a defect found in one run can be reproduced and verified after the fix.

3. Incomplete Control Flow Graph in White-Box Testing

The developer's control flow graph is faulty because it omits decision nodes and loops. A missing decision node means that one of the branches of the code is not represented, so no test is planned for it; a missing loop means that the back edge from the end of the loop body to the loop condition is absent, so the graph looks like straight-line code and the zero-iteration, single-iteration and multiple-iteration behaviours are never distinguished. As a result, the calculated cyclomatic complexity is lower than the actual value, fewer independent paths are identified, and the path coverage achieved is poor even though the tests appear complete according to the graph.

The root causes are that the graph was drawn from memory or from the design rather than from the actual code, compound conditions such as (a > 0 && b > 0) were treated as one node instead of being split into separate decisions, the false branches and the loop-back edges were forgotten, and there was no review or tool support to check the graph against the code, using V(G) = E - N + 2.

A proper representation is built by assigning one node to each sequential block and one to each decision, joining the nodes by edges for both true and false outcomes, and adding a back edge for each loop. For example, a module with a while loop containing one if-else and a final decision has nodes for initialisation, the loop condition, the if test, the true block, the false block, the loop increment, the final decision, two outcome blocks and the exit; with 12 edges and 10 nodes, V(G) = 12 - 10 + 2 = 4. The four independent paths are: loop not entered; loop entered with the true branch; loop entered with the false branch; and multiple iterations with the final decision false. One test case is designed for each path. Improved path testing ensures better coverage because it guarantees that every statement and every branch is executed at least once, and it

exposes logic defects such as wrong conditions, unreachable code and loop errors, which are hidden when the graph is incomplete.

4. 100% Statement Coverage but Logical Errors Undetected

The testing performed is inadequate even though the report shows 100% statement coverage. Statement coverage only proves that every line of code was executed at least once; it does not show that every decision was evaluated to both true and false, that each condition inside a compound expression was tested, or that every combination of paths was taken. The gaps in the coverage criteria are therefore branch (decision) coverage, condition coverage and path coverage.

Statement coverage was insufficient because of the following reason. In the code if (x > 10 && y < 5) { total = total + bonus; }, a single test with x = 15 and y = 3 executes all the statements, so the coverage is 100%, but the false outcome of the decision is never exercised, and the case in which the code incorrectly omits or wrongly evaluates the condition is not caught. An else path which contains no statement, an incorrectly written operator (such as > in place of >=), or a wrong logical operator (AND in place of OR) all remain undetected, since the statement coverage metric does not require the false path to be executed.

The recommended improved strategy is to use branch coverage, so that every decision is tested both true and false; condition coverage (or modified condition/decision coverage) so that each atomic condition independently takes true and false; and basis path testing based on the cyclomatic complexity so that all independent paths are executed. For the above example, the tests would be (x = 15, y = 3), (x = 5, y = 3), (x = 15, y = 8) and the boundary values x = 10, 11 and y = 4, 5. This approach enhances test effectiveness because every outcome of every decision is verified, so logical errors in conditions are revealed, and the coverage figure becomes a more reliable measure of the real quality of the testing.

5. Age Validation Field (0-100) with Only Two Classes

The tester's partitioning with only the classes Negative numbers and Positive numbers has several errors. The classes are based on the sign of the number instead of on the validity rule of the field, so the valid range 0 to 100 is not isolated from the invalid values above 100; the positive class contains both valid ages (such as 25) and invalid ages (such as 150), which violates the principle that all members of a class must behave in the same way. Zero, which is valid, does not fit properly in either class, and the class of non-numeric, decimal and empty input is completely absent. Boundary Value Analysis was also not used, so the edges of the range at 0 and 100 and the values just outside them, -1 and 101, were not tested, which are the places where defects such as using < instead of <= appear.

The corrected partitions are: the valid class, 0 to 100; invalid class 1, below the range, that is any value less than 0; invalid class 2, above the range, that is any value more than 100; invalid class 3, non-integer values, including decimals such as 25.5; and invalid class 4, non-numeric or empty input, including letters, special

characters and blank. Representative tests are 45 (accepted), -10 (rejected), 150 (rejected), 25.5 (rejected), "abc" (rejected) and blank (rejected). The boundary values are -1, 0, 1, 99, 100 and 101, with expected results of reject, accept, accept, accept, accept and reject.

These partitions ensure more robust input validation because each class now corresponds to a single expected behaviour and error message, every kind of invalid input is detected, and the boundary tests prove that the limits are implemented exactly as specified. The number of tests stays small, but the coverage of the input domain is complete and the defect detection capability is considerably higher than that of the original two-class design.

6. Condition Coverage for a Complex if Statement (A && B ...)

The question text in the supplied document is cut off after "if(A && B"; therefore, this answer assumes the common form of the problem, a compound decision such as if ((A && B) || C), and the same reasoning applies to any similar compound condition. The developer's claim of full condition coverage is not sufficient evidence of adequate testing. Condition coverage requires only that each atomic condition (A, B and C) evaluates to both true and false at least once. It does not require that the decision as a whole evaluates to both true and false, and it does not show that each condition independently affects the outcome.

A simple illustration uses the decision if (A && B). The two tests (A = T, B = F) and (A = F, B = T) make each condition take both true and false, so condition coverage is 100%, yet the decision evaluates to false in both tests and the true branch is never executed. In the same way, for (A && B) || C, faults such as replacing && by || or ignoring condition B are not exposed unless each condition is shown to change the outcome on its own.

The corrective approach is Modified Condition/Decision Coverage (MC/DC) or, at least, decision and multiple-condition coverage. For (A && B) || C, MC/DC needs n + 1 = 4 tests: (T, T, F) giving true; (T, F, F) giving false; (F, T, F) giving false; and (F, F, T) giving true. In pairs, each test shows that changing one condition alone changes the outcome. The approach is effective because it gives strong fault detection with only linear growth in the number of tests (n + 1, instead of 2 to the power n for exhaustive combinations), and it proves that every condition is necessary and correctly implemented.

7. Decision Table for a Discount System (Premium Member and Coupon)

The decision table is incomplete because several combinations of the conditions Premium Member and Coupon Applied were not included, so the rules for those combinations were never tested and the pricing logic for them was never verified. A decision table with n binary conditions must have 2 to the power n rules; the team probably listed only the obvious cases (premium with coupon, and regular without coupon) and left out the mixed ones, such as a non-premium member with a coupon and a premium member without a coupon, or

defined the same action for different conditions without checking. The errors in the construction include missing rules, no check for contradictory or redundant rules, and no defined action for certain columns.

The root cause is that the table was built from a quick reading of the requirement rather than by systematic enumeration of all condition combinations, and no review was performed to confirm that the rules were complete and consistent. Because the discount depends on the interaction of the conditions (for example, whether the coupon discount is added to the member discount or replaces it, or whether the highest discount applies), the missing combinations lead directly to the price calculation errors seen.

The revised exhaustive table, assuming a member discount of 10% and a coupon discount of 5%, has four rules for two conditions. Rule 1: Premium = Yes, Coupon = Yes, giving the combined discount as defined by the business rule (for example 15%, or the higher of the two). Rule 2: Premium = Yes, Coupon = No, giving 10%. Rule 3: Premium = No, Coupon = Yes, giving 5%. Rule 4: Premium = No, Coupon = No, giving 0%. If a coupon validity condition (valid or expired) is added, the table grows to eight rules, with expired coupons giving no discount and an error message. Each rule has one test case, and boundary values for the order amount, such as the minimum amount needed for the coupon, are tested for each rule. The table prevents production defects because all combinations are covered and verified against explicit expected prices, ambiguity in the business rules is resolved during construction, and no combination of conditions remains untested.

8. Loop Testing with Only n = 1 and n = max

The tester's loop testing strategy is incomplete because testing only the first and last iterations cannot reveal faults at the extremes of the loop's control variable. The loop that is skipped entirely (n = 0) tests whether the program correctly handles an empty input, and the iteration beyond the maximum (n = max + 1) tests whether the loop terminates correctly and the program resists overflow or an array index out of range. Defects such as a wrong loop condition (using <= instead of <), an uninitialised counter, an infinite loop, and the off-by-one error are exactly the defects at these points, and so they were missed.

The root causes are that the tester considered only the typical and the maximum case, assumed that the case of no iteration is not a realistic input, did not analyse the loop condition and its boundary, and did not follow a loop testing technique. The common belief that the loop works for the first and the last iterations and therefore works for all is wrong, because the behaviour at the entry and exit boundaries is controlled by the condition and not by the body of the loop.

The corrected approach for a simple loop with a maximum of n passes is to test: skipping the loop entirely (0 passes); one pass; two passes; a typical number of passes m, where m is less than n; n - 1 passes; n passes; and n + 1 passes (which must be rejected or handled properly). For nested loops, the innermost loop is tested first using the simple loop tests while the outer loops are held at their minimum values; then testing works outward,

one loop at a time, with the other loops kept at typical values, and finally the combinations of the minimum and maximum values for the nested loops are tested. For concatenated loops that are independent, each is tested as a simple loop, and if they are dependent, they are treated as nested loops. This approach ensures path integrity because every distinct way of entering, repeating and leaving the loop is executed, the number of tests is kept manageable, and the boundary defects are exposed.

9. Error Guessing for a Safety-Critical Braking System

Relying only on error guessing for a safety-critical braking system is a serious error in strategy. Error guessing depends on the experience and the intuition of the individual tester; it has no defined starting point, no systematic coverage measure, and no way to show that all the requirements, input ranges and system states have been tested. The results are not repeatable and cannot be audited, which is unacceptable for a system that safety standards such as ISO 26262 require to be verified with documented, traceable, structured methods. Defects that are not obvious to the tester, particularly those at boundaries or in rare sequences of states, are likely to remain undetected.

The impact on test adequacy is large. The coverage of requirements is unknown, boundary values such as the threshold of the pedal pressure or the wheel speed may be missed, state-dependent behaviour such as the transition from normal to ABS braking or to the failure mode may be untested, and a false confidence in the safety of the system is produced. An undetected defect in such a system can cause an accident, so the residual risk is unacceptable.

The strategy should be corrected by combining Boundary Value Analysis and State Transition Testing, with error guessing retained only as a supplement. BVA is applied to the inputs, for example pedal pressure, vehicle speed, wheel-slip ratio and sensor voltage, with tests at the minimum, minimum plus one, nominal, maximum minus one, maximum and just outside the range. State Transition Testing models the states Idle, Normal Braking, ABS Active, Emergency Braking and Fault/Fail-safe, and the test cases cover every valid transition (for example, Normal to ABS when the slip exceeds the threshold) and invalid transitions (for example, ABS Active to Idle while the pedal is pressed), including sequences of transitions and sensor-failure events. The combined approach increases safety assurance because it provides documented and measurable coverage of the input domain and of all the states, it verifies safe behaviour at the critical limits, and it makes the results traceable and repeatable for certification, while the experience of the testers is used to add further tests for hazards that the models do not capture.

10. CFG Without Else Branches of Nested if Statements

The graph is wrongly constructed because the tester drew only the nodes and edges along the path where each if condition is true and omitted the false (else) outcomes of the nested decisions. In a correct CFG each decision node must have two outgoing edges, one for the true and one for the false outcome, even if the else part is

empty and merely leads to the join node. By omitting these edges, the false-branch paths are not visible, and the error-handling paths that execute when a condition fails are never targeted by test cases. The tests executed all the statements on the true paths, which gives 100% statement coverage, but since the false edges were not in the graph, none of them was exercised, giving 0% branch coverage on the error paths.

The root causes of the missing nodes are the assumption that an if statement without an else clause has no else branch to be tested, the habit of drawing the graph only for the successful flow, the lack of an explicit join node where the true and false branches merge, and the absence of a check of the number of edges against the cyclomatic complexity calculated from the code. Nested decisions made the omission worse because each missing outer else removes all the inner decisions that depend on it.

The corrected CFG is constructed as follows. For the code if (A) { if (B) { S1 } else { E2 } } else { E1 }, the nodes are: N1 (decision A), N2 (decision B), N3 (S1), N4 (E2, the error path for B false), N5 (E1, the error path for A false) and N6 (the join or exit node). The edges are N1 to N2 (A true), N1 to N5 (A false), N2 to N3 (B true), N2 to N4 (B false), and N3, N4 and N5 each to N6, so E = 7 and N = 6, giving V(G) = 7 - 6 + 2 = 3. The three independent paths are: P1 = N1, N2, N3, N6 (A true, B true); P2 = N1, N2, N4, N6 (A true, B false); and P3 = N1, N5, N6 (A false). The test cases are A = T, B = T; A = T, B = F; and A = F. These tests ensure branch adequacy because every outcome of every decision, including each error path, is executed at least once, so the branch coverage becomes 100% and the faults in the error handling are detected.

Code of Conduct:

I R.RACHITHA/192421058 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand